

# Service Communication Diagram

University ERP · Advanced Software Engineering · UCSC

## 1. Purpose

This document describes how the University ERP microservices communicate with each other and with the client. Two communication patterns are used: synchronous REST (HTTP/JSON via axios) for operations that require an immediate response, and asynchronous Kafka events for decoupled state propagation. This revision separates two views that must not be read as the same thing: the diagram in Section 2 is a target/future-state design (single API gateway, purely event-driven backend); Sections 4 and 5 describe the system as it is actually built and running today, which differs from that target in three specific ways summarised in Section 3.6. Of the eight originally designed services, six are built and running (Auth, Student, Exam, Result, Transcript, Notification); Course Service and Reporting Service remain design-only in both views.

## 2. Diagram

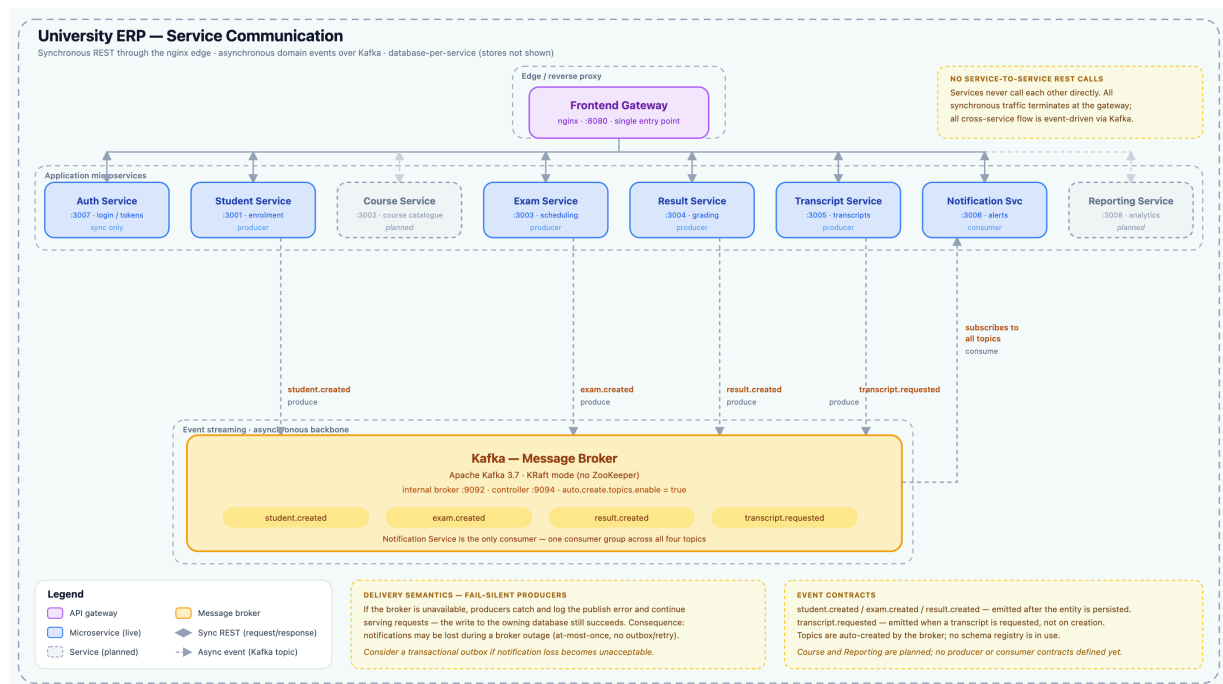


Figure 1 — Target / future-state architecture: a single API gateway in front of every service, with cross-service coordination handled entirely through Kafka events and no direct service-to-service REST calls. This is a design goal, not the current build — Section 3.6 lists exactly how the running system differs from this picture today.

## 3. Communication Layers

### 3.1 Client Layer

The system has a single external client: a browser-based single-page application (Bootstrap 5 + jQuery) served on port 8080. The frontend adapts its UI by role (STUDENT, LECTURER, HOD, EXAM\_DIVISION, ADMIN) rather than being separate Mobile App / Admin Portal clients — those were part of the original design vocabulary but were never built as distinct applications.

### 3.2 NGINX — Static Frontend Only (Corrected)

In the running system, NGINX (nginx:alpine, docker-compose service "frontend") serves only the static SPA files on port 8080 — its nginx.conf contains a single `try_files` location block for the SPA plus a caching rule for JS/CSS. It performs no path-based API routing, authentication, or request transformation. Each of the six built services publishes its port directly to the host (3007, 3001, 3003, 3004, 3005, 3006), and the frontend JavaScript calls each service directly at `http://localhost:<port>`. The original design intent — NGINX as a true API gateway routing `/api/<resource>` by path prefix — is retained in Section 6 as a documented but unbuilt goal.

### 3.3 Internal Service Network

The three real cross-service calls (Exam → Student, Transcript → Student, Transcript → Result) run over the `erp-network` Docker bridge using service names as DNS hostnames (e.g., `http://student-service:3001`). JWT tokens are validated by each service's own middleware — no request is forwarded to Auth Service for authentication at runtime.

### 3.4 MongoDB Data Layer

Each built service connects exclusively to its own logical database on a single shared `mongo:7` container (not a multi-node cluster). No service queries another service's database. Transcript Service owns no database at all — it aggregates data on demand via REST calls to Student Service and Result Service. `courses_db` and `reports_db` are not provisioned, since Course Service and Reporting Service do not exist.

### 3.5 Kafka Event Bus

The async event bus is fully implemented and running (Apache Kafka 3.7, KRaft mode) — this is no longer a design-only element. Producers emit events and continue immediately without waiting for consumers; if Kafka is temporarily unreachable, producers fail silently and simply skip the publish rather than blocking the request. Only Notification Service currently consumes events; it retries topic subscription every 10 seconds until topics are auto-created by the first publish.

### 3.6 Target Design vs. As-Built — Key Differences

The diagram in Section 2 intentionally shows where this architecture could go next, not where it is today. Three concrete differences between that target design and the running system:

- **Gateway:** target design routes all client traffic through a single NGINX API gateway. As built, NGINX serves only static frontend files and the browser calls each service's port directly — see Section 3.2.
- **Service-to-service calls:** target design assumes zero direct REST calls between services, with all cross-service coordination event-driven. As built, three direct synchronous REST calls exist and are load-bearing: Exam → Student, Transcript → Student, Transcript → Result — see Section 4.
- **Kafka topics:** target design uses student.created, exam.created, result.created, and a new transcript.requested topic — implying Transcript Service would become an event consumer instead of calling Student and Result services synchronously. As built, the real topics are student.created, exam.registered, and result.published; Transcript Service is not a Kafka producer or consumer at all — see Section 5.

Moving to the target design would be a genuine architecture change (a real gateway, an event-driven Transcript Service, new topic contracts) — not just a documentation update. See Section 8 for this framed as a next-iteration item.

## 4. Synchronous REST Flows — As Built

Solid arrows in the diagram represent synchronous HTTP/JSON calls using axios. The caller blocks and waits for a response before proceeding.

### 1. Browser SPA → Each of the 6 built services, directly (no gateway)

- **Endpoint:** http://localhost:<port>/api/...
- **Trigger:** Any user action in the frontend
- **Purpose:** Direct REST call to the owning service
- **Error handling:** Handled per-service; no shared gateway behaviour

### 2. Exam Service → Student Service

- **Endpoint:** GET /api/students/:id
- **Trigger:** POST /api/exams/:id/entries
- **Purpose:** Verify the student exists before creating an Entry
- **Error handling:** 404 from Student Service → Exam Service returns 404 to client; Student Service down → 502 Bad Gateway

### 3. Transcript Service → Student Service

- **Endpoint:** GET /api/students/:id
- **Trigger:** GET /api/transcripts/:studentId/semester/:sem or /final
- **Purpose:** Fetch student profile for the transcript
- **Error handling:** Student Service down → 502 Bad Gateway

#### 4. Transcript Service → Result Service

- **Endpoint:** GET /api/results?... and GET /api/results/gpa/:studentId
- **Trigger:** GET /api/transcripts/:studentId/semester/:sem or /final
- **Purpose:** Fetch results and GPA for the transcript
- **Error handling:** Result Service down → 502 Bad Gateway

### 5. Asynchronous Kafka Flows — As Built

Dashed arrows in the diagram represent asynchronous Kafka events, fully wired and running against the Kafka broker. The producing service emits an event and continues immediately without waiting for any consumer.

#### 5.1 Events

##### Event: student.created

|                           |                                                                                                   |
|---------------------------|---------------------------------------------------------------------------------------------------|
| <b>Producer</b>           | Student Service                                                                                   |
| <b>Consumer(s)</b>        | Notification Service                                                                              |
| <b>Trigger</b>            | Successful POST /api/students                                                                     |
| <b>Payload</b>            | { "studentId": "...", "name": "...", "email": "...", "_id": "..." }                               |
| <b>Consumer Action(s)</b> | Persisted as a NotificationLog document (audit trail); no real email/SMS is sent in the prototype |

##### Event: exam.registered

|                           |                                                                                                     |
|---------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Producer</b>           | Exam Service                                                                                        |
| <b>Consumer(s)</b>        | Notification Service                                                                                |
| <b>Trigger</b>            | Successful POST /api/exams/:id/entries                                                              |
| <b>Payload</b>            | { "studentId": "...", "studentName": "...", "examTitle": "...", "examId": "...", "entryId": "..." } |
| <b>Consumer Action(s)</b> | Persisted as a NotificationLog document (audit trail)                                               |

##### Event: result.published

|                           |                                                                                                 |
|---------------------------|-------------------------------------------------------------------------------------------------|
| <b>Producer</b>           | Result Service                                                                                  |
| <b>Consumer(s)</b>        | Notification Service only                                                                       |
| <b>Trigger</b>            | PATCH /api/results/gpa/:gpald/finalize                                                          |
| <b>Payload</b>            | { "studentId": "...", "studentRegNo": "...", "academicYear": 1, "gpa": 3.7, "status": "final" } |
| <b>Consumer Action(s)</b> | Persisted as a NotificationLog document (audit trail)                                           |

Note: this corrects earlier drafts of this document, which described a simple grade/score payload and listed Reporting Service and Transcript Service as consumers. Neither is a Kafka consumer in the built system — Reporting Service does not exist, and Transcript Service fetches results synchronously via REST on demand rather than maintaining a Kafka-fed local cache. A built Reporting Service would be the natural fourth consumer of result.published in a future iteration.

## 6. NGINX Path Routing — Designed, Not Implemented

The table below is the original design intent for NGINX as a true API gateway. None of it is active in the running system (see Section 3.2) — it is retained here as the target design for a future iteration, alongside the two services that would need to exist for the last two rows to be meaningful. This table predates the redrawn diagram in Section 2; both describe an unbuilt API-gateway pattern, this one at path-prefix granularity, the newer one as a single simplified gateway box.

| External Path Prefix | Upstream Service     | Internal Port | Notes                                |
|----------------------|----------------------|---------------|--------------------------------------|
| /api/auth            | Auth Service         | 3007          | Designed; not routed via NGINX today |
| /api/students        | Student Service      | 3001          | Designed; not routed via NGINX today |
| /api/courses         | Course Service       | 3002          | Designed; service itself not built   |
| /api/exams           | Exam Service         | 3003          | Designed; not routed via NGINX today |
| /api/results         | Result Service       | 3004          | Designed; not routed via NGINX today |
| /api/transcripts     | Transcript Service   | 3005          | Designed; not routed via NGINX today |
| /api/notifications   | Notification Service | 3006          | Designed; not routed via NGINX today |
| /api/reports         | Reporting Service    | 3008          | Designed; service itself not built   |

## 7. Error Propagation in Sync Calls

Because the three real synchronous cross-service calls (Section 4, rows 2–4) introduce a dependency chain, each calling service must handle upstream failures explicitly. The following rules are applied — these hold regardless of the gateway question in Section 3.2, since they govern service-to-service calls, not client-to-service calls:

- If the upstream service returns 404: the calling service returns 404 to the client with a meaningful error message indicating which entity was not found and in which service.
- If the upstream service is unreachable (connection refused, timeout): the calling service returns 502 Bad Gateway with a message indicating which upstream service failed.
- If the upstream service returns 5xx: the calling service returns 502 Bad Gateway.
- The JWT from the original client request is forwarded to the upstream service in the Authorization header, so the upstream service can also authenticate the internal call.
- Async Kafka consumers are not part of a synchronous chain — if a consumer fails to process an event, Kafka retains the event and the consumer retries. This does not affect the original HTTP response.

## 8. Current Implementation Status

Six of the eight designed services are built and running: Auth, Student, Exam, Result, Transcript, and Notification. All three sync REST flows in Section 4 (rows 2–4) are fully implemented and operational. The Kafka event pipeline (Section 5) is fully wired and running against a live Kafka broker — this corrects earlier drafts of this document, which stated Kafka event emission was "noted in code comments but not wired to a running Kafka instance." Course Service and Reporting Service remain designed but not implemented — no directory under services/, no Dockerfile, and no entry in docker-compose.yml. NGINX does not perform path-based API routing in the current build (Section 3.2); this differs from the original design intent documented in Section 6. The gap between this as-built state and the target design in Section 2/3.6 — a real API gateway, elimination of direct service-to-service REST calls, and an event-driven Transcript Service — is the natural scope for the next iteration of this system, not a discrepancy to hide.

References: Service Identification Table, Bounded Context Diagram, API Endpoint Design, Architecture Design Document, Section 7 (Inter-Service Communication).